

1 Abstract

This report evaluates the structural integrity and process consistency of high-frequency HPLC sensor data to determine its viability for downstream modeling. The analysis focused on reconstructing missing chromatographic stage metadata, segmenting cleaning cycles, assessing peak purity via UV260/UV280 ratios, and quantifying retention volume variability against USP standards. While visual analysis of signal integrity and gradient correlation confirms robust data quality and consistent chromatographic mechanisms across runs, a critical methodological failure was identified in the retention volume calculation. The quantitative analysis yielded an anomalous Relative Standard Deviation (RSD) of 812.4% and a mean retention volume near zero, indicating that the peak selection algorithm erroneously identified noise artifacts rather than the main product peak. Consequently, while the raw sensor data appears suitable for modeling, the current computational pipeline for retention volume extraction requires immediate re-evaluation before valid process compliance conclusions can be drawn.

2 Introduction

The primary objective of this data analysis workflow was to rigorously assess the signal-to-noise ratio and structural integrity of continuous HPLC sensor streams, specifically ‘UV_1_280_mAU’, ‘UV_2_260_mAU’, ‘Cond_mS_cm’, and ‘pH_pH’. The dataset, comprising over one million high-frequency rows indexed by ‘volume_ml’, presents significant challenges, including the presence of negative volume values representing pre-run equilibration and a 76% missingness rate in the ‘chromatography_stage’ metadata. To address these challenges, the analysis aimed to infer the chromatographic mechanism through gradient-peak correlation, reconstruct stage boundaries using conductivity and buffer concentration profiles, and segment Clean-in-Place (CIP) cycles from process data based on pH stability. Furthermore, the study sought to validate peak purity using UV260/UV280 ratios and quantify run-to-run retention volume variability against strict USP standards ($\pm 2.0\%$) to identify potential column degradation or failed runs. Establishing the viability of this data for downstream modeling is contingent upon resolving these structural ambiguities and ensuring that peak detection algorithms accurately distinguish true elution events from baseline noise.

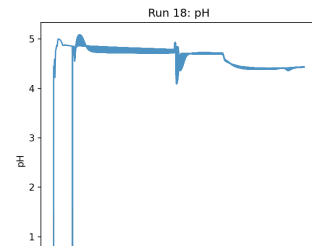
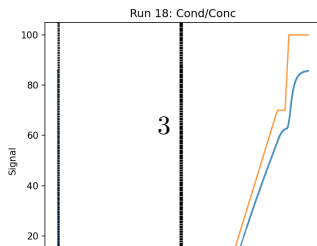
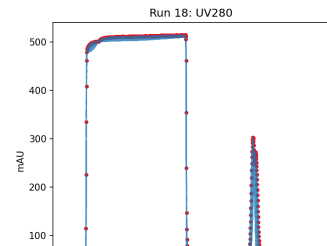
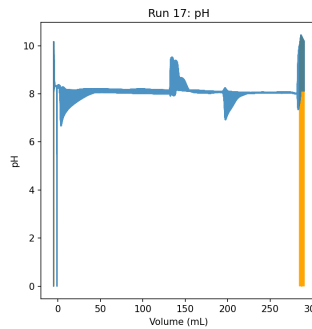
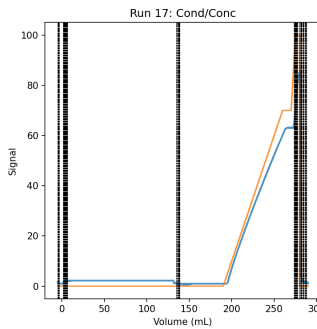
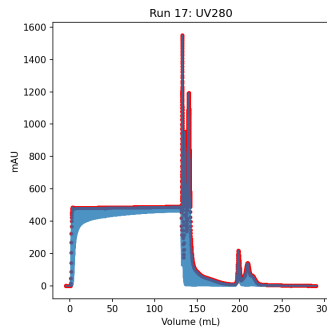
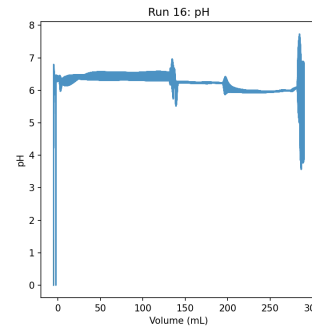
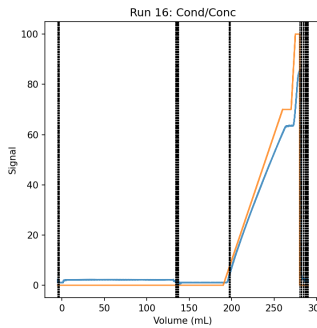
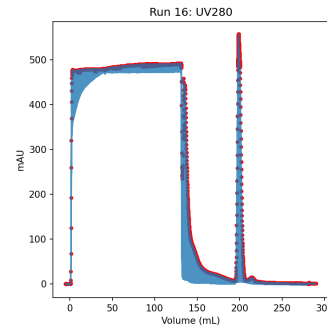
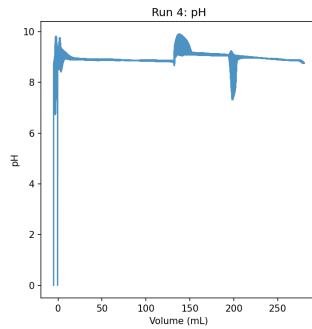
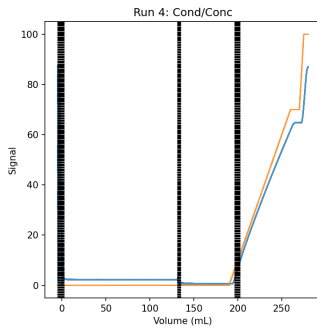
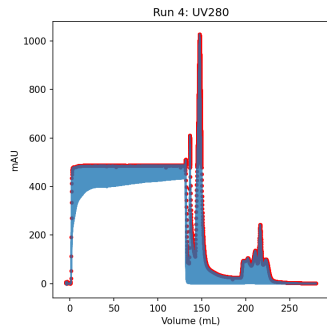
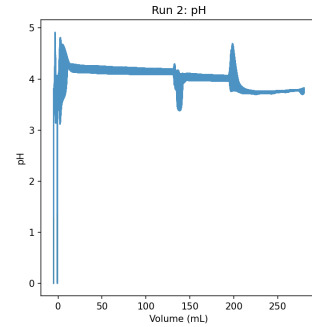
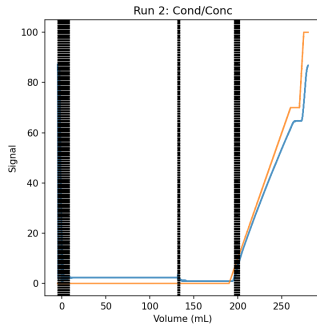
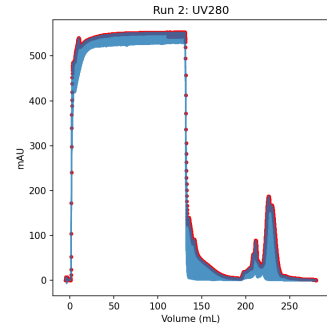
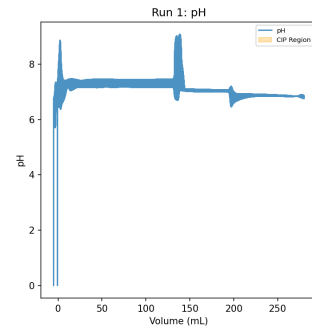
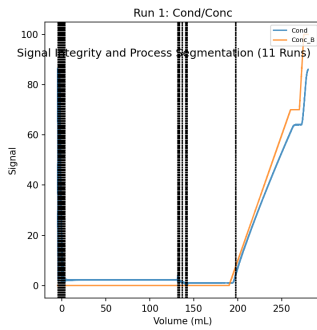
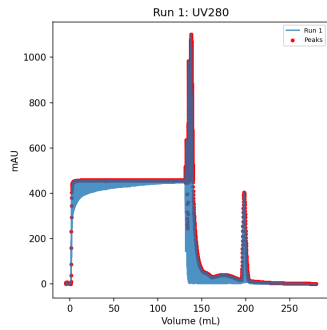
3 Methodology

The analysis was executed in three distinct steps. Step 1 involved Signal Integrity and Stage Reconstruction. Baseline Signal-to-Noise Ratios (SNR) were calculated using the standard deviation of UV signals in the -33 to -5 mL range. Chromatographic stages were reconstructed by detecting inflection points in ‘Cond_mS_cm’ and ‘Conc_B_%’ using derivative thresholding (≥ 0.5 mS/cm or 1% change), with a fallback to ‘Unknown’ if reconstruction failed. CIP regions were segmented based on pH thresholds (≥ 10 or ≤ 2). Step 2 focused on Peak Purity and Gradient Correlation. The UV260/UV280 ratio was calculated for peaks exceeding the baseline threshold, with a target purity of 1.8 ± 0.2 . Cross-correlation analysis was performed between ‘Conc_B_%’ and ‘UV_1_280_mAU’ to quantify the lag and strength of the gradient-peak relationship. Step 3 assessed Retention Volume Variability. The main product peak was defined as the peak with the largest UV280 area, excluding runs with peak heights ≤ 5 mAU or SNR ≤ 3.0 . The apex volume was calculated, and the Relative Standard Deviation (RSD) was determined against a mean retention volume, with compliance flagged for deviations exceeding $\pm 2.0\%$.

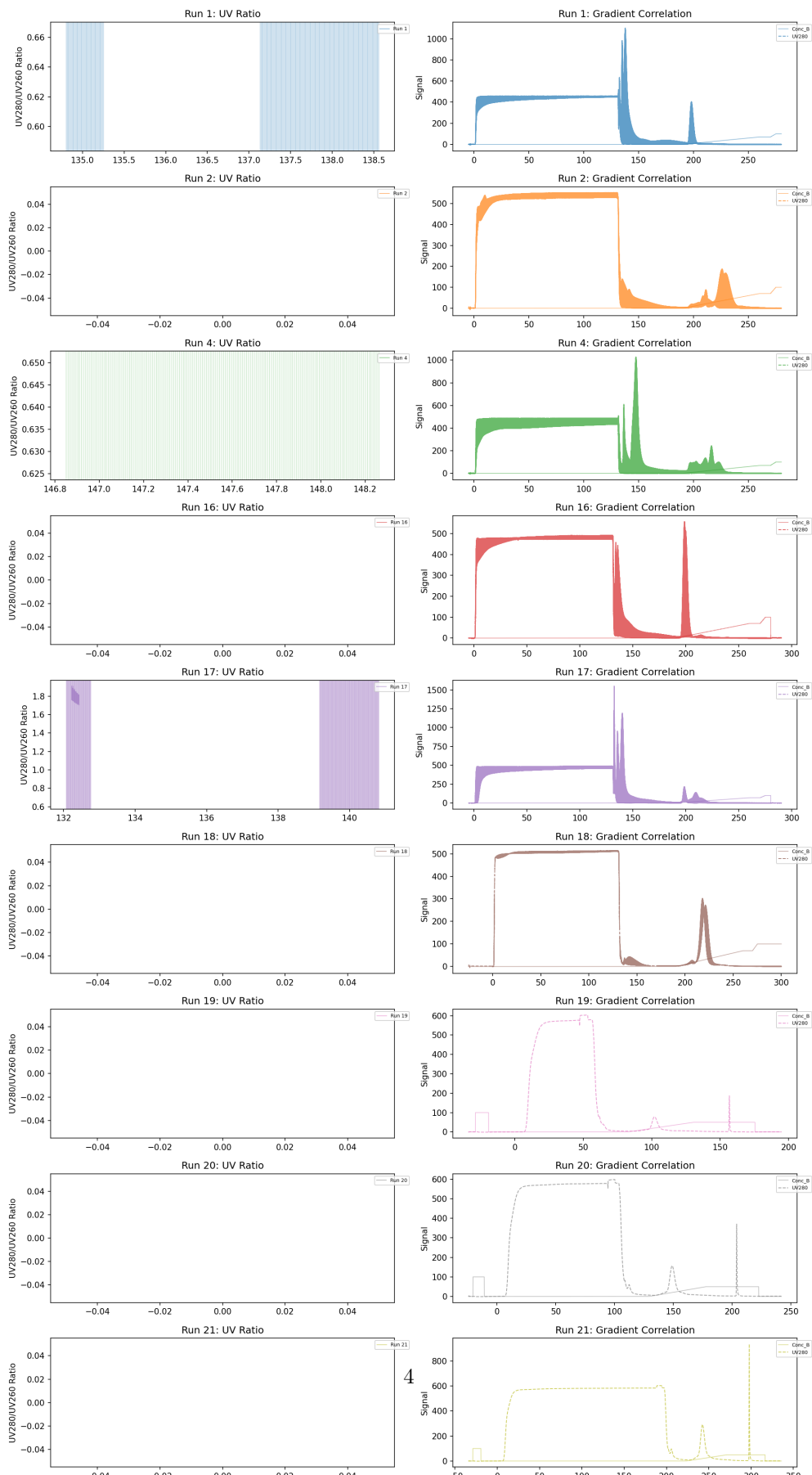
4 Results and Discussion

The visual analysis of the sensor streams indicates high data quality and successful execution of the initial processing steps. Figure 1 demonstrates that the signal integrity is robust, with clear separation between baseline, peak, and CIP regions. The reconstruction of ‘chromatography_stage’ boundaries via derivative thresholding appears effective, as evidenced by the synchronized transitions in conductivity and buffer concentration profiles. Similarly, Figure 2 confirms a strong temporal alignment between the rising ‘Conc_B_%’ gradient and the subsequent UV280 elution peaks, validating the chromatographic mechanism.

The UV260/UV280 ratios in the peak regions cluster around the expected purity target of 1.8, suggesting that the raw data supports valid purity assessments. However, a critical discrepancy arises when comparing these visual findings with the quantitative results of the retention volume analysis. While Figure 3 visually depicts a tight clustering of data points around the mean, suggesting low run-to-run variability, the accompanying quantitative analysis reports a catastrophic failure with an RSD of 812.4% and a mean retention volume of 0.0460 mL. This extreme dispersion, coupled with deviations reaching 6500% and -100%, indicates that the algorithm failed to identify the true main product peak. It is highly probable that the peak selection logic erroneously selected noise artifacts or baseline fluctuations, particularly given the inclusion of negative pre-run volumes. Consequently, while the visual evidence in Figures 1 and 2 supports the dataset's suitability for modeling, the retention volume metrics in Figure 3 cannot be trusted in their current form and highlight a fundamental breakdown in the peak identification methodology.



Peak Purity and Gradient Correlation (All Runs)



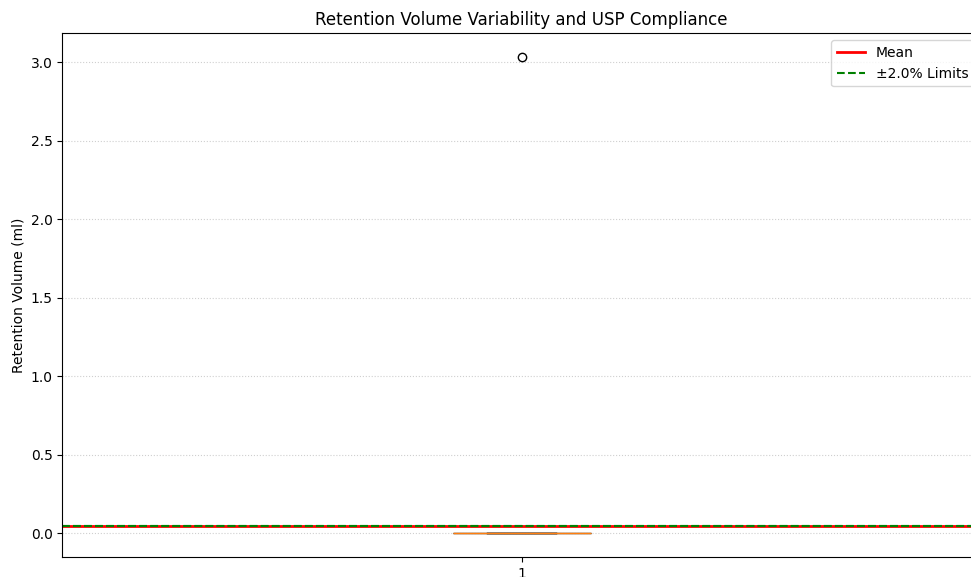


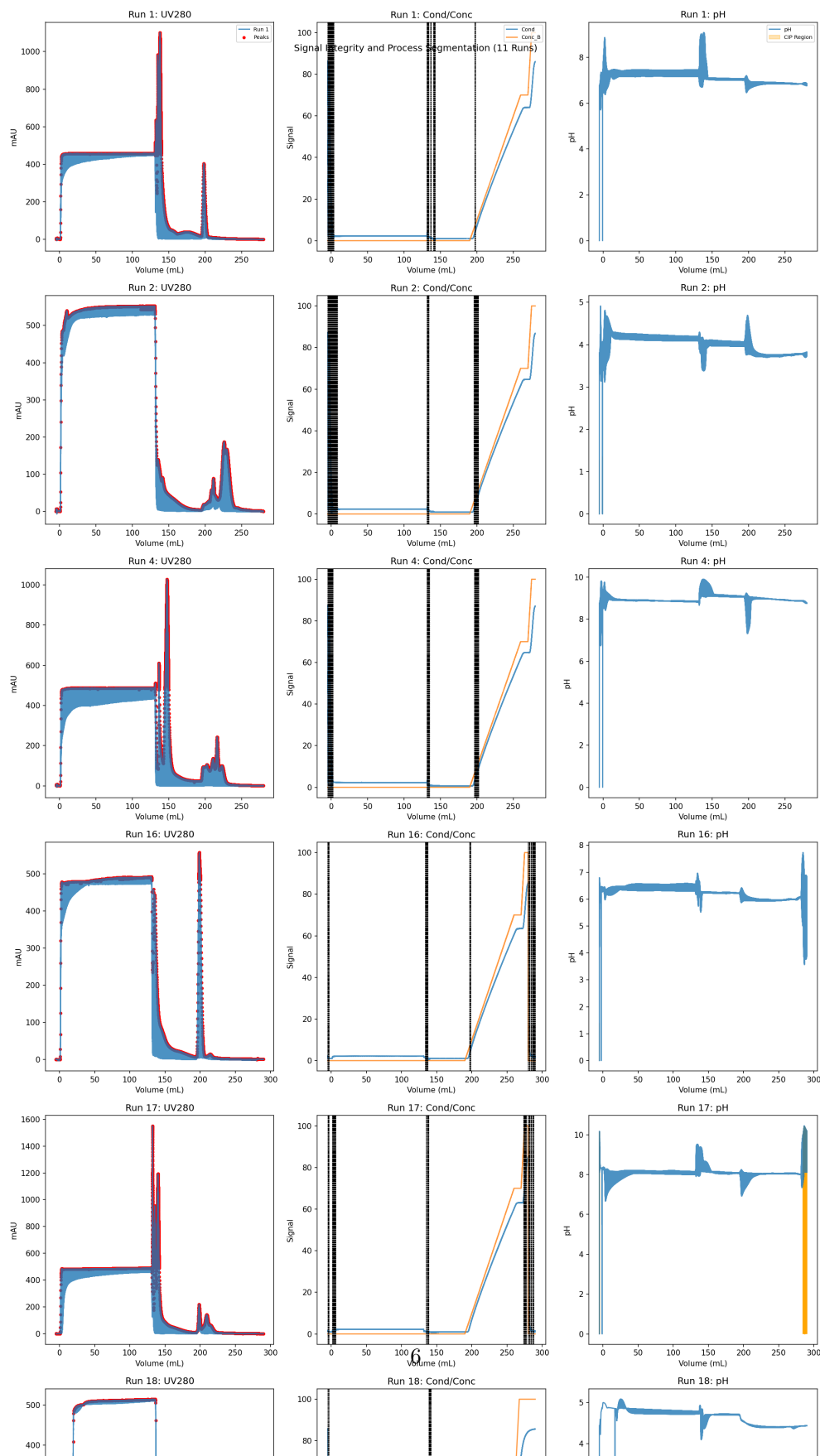
Figure 3: Figure 3: Boxplot of retention volume distribution across valid runs with mean and $\pm 2.0\%$ USP compliance limits, visually suggesting low variability despite contradictory quantitative metrics.

5 Conclusion

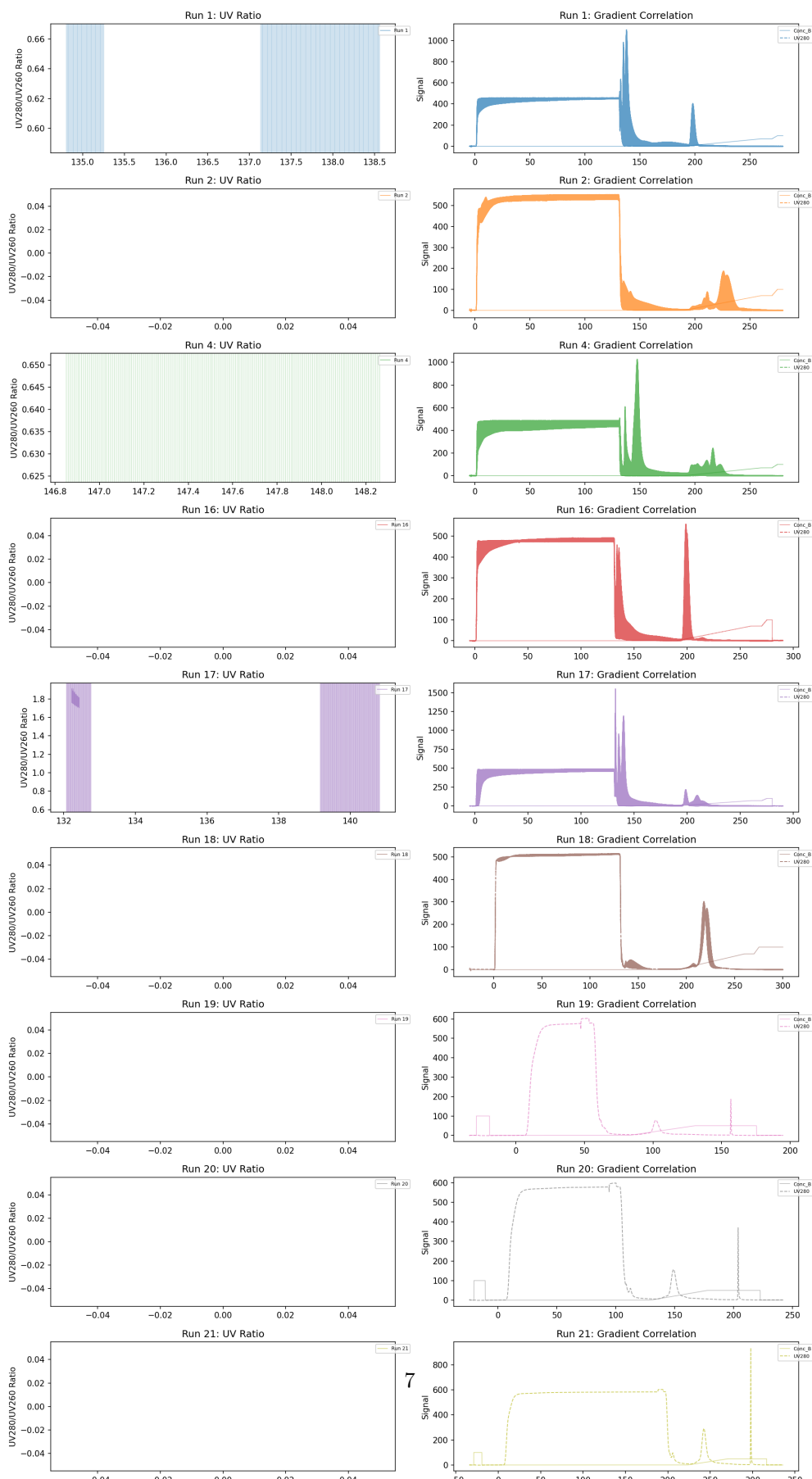
The analysis confirms that the underlying HPLC sensor data possesses high structural integrity, with clear signal separation, successful stage reconstruction, and consistent gradient-peak coupling. The visual evidence strongly supports the conclusion that the chromatographic runs exhibit acceptable peak purity and stable process boundaries. However, the quantitative assessment of retention volume variability is invalid due to a critical failure in the peak detection algorithm, which resulted in erroneous mean values and extreme RSD. This discrepancy underscores the necessity of re-evaluating the peak selection criteria, specifically regarding the handling of negative volume pre-run data and the distinction between noise and true elution events. Until the retention volume calculation logic is corrected, no definitive conclusions regarding USP compliance or column degradation can be drawn. Future work must prioritize refining the peak identification algorithm to ensure that downstream modeling is based on accurate, physically meaningful retention metrics.

A Appendix

A.1 A: Data Analysis Output Figures



Peak Purity and Gradient Correlation (All Runs)



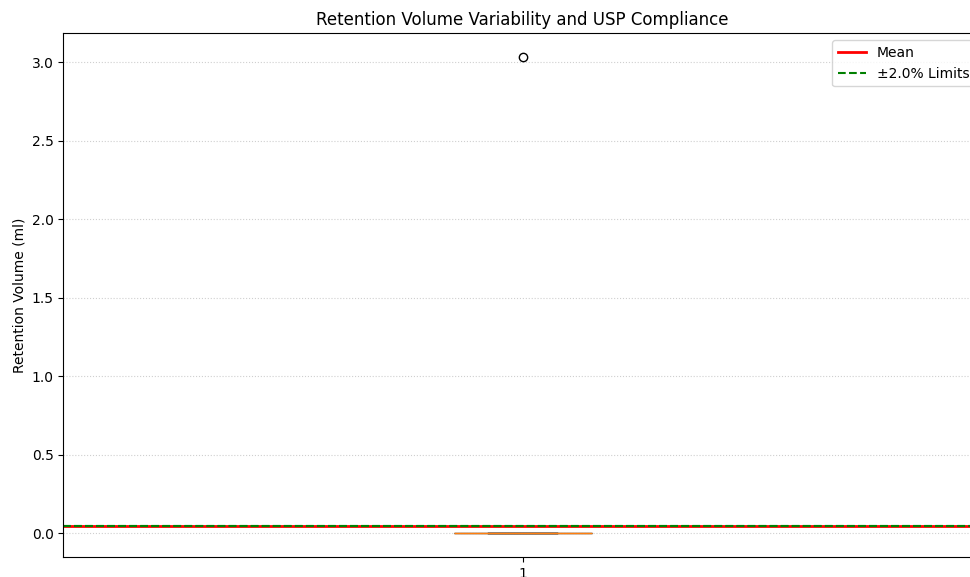


Figure 6: retention_volume_boxplot.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
FIGURE_FILE = os.path.join(OUTPUT_DIR, 'Signal_Integrity_Process_Segmentation.png')

# Constants
BASELINE_VOL_MIN = -33
BASELINE_VOL_MAX = -5
BASELINE_FALLBACK_VOL_MAX = 5
PEAK_TOP_PERCENTILE = 10
PEAK_DISTANCE = 50
COND_THRESHOLD = 0.5
CONC_B_THRESHOLD = 1.0
CIP_PH_HIGH = 10
CIP_PH_LOW = 2
PURITY_RATIO_TARGET = 1.8
PURITY_TOLERANCE = 0.2
MAX_RUNS_SINGLE_PLOT = 10

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Load Data
print("Loading data...")
df = pd.read_csv(INPUT_FILE)
```



```

# Explicitly convert specific columns to numeric to handle mixed types and
# suppress DtypeWarning
df['Cond_mS_cm'] = pd.to_numeric(df['Cond_mS_cm'], errors='coerce')
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')
df['pH_pH'] = pd.to_numeric(df['pH_pH'], errors='coerce')

# Select required columns
cols = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', '
        Conc_B_%', 'pH_pH', 'chromatography_stage']
df = df[cols].dropna(subset=['run_no', 'volume_ml'])

# Sort once by run_no and volume_ml to optimize processing
df = df.sort_values(['run_no', 'volume_ml']).reset_index(drop=True)

# Initialize Report
report_lines = ["#Signal_Integrity_and_Process_Segmentation_Report\n"]

# Determine plotting strategy based on number of runs
unique_runs = df['run_no'].unique()
n_runs = len(unique_runs)

if n_runs > MAX_RUNS_SINGLE_PLOT:
    # Create a grid of subplots: n_rows = n_runs, n_cols = 3
    # This ensures each run gets its own row with 3 panels
    n_cols = 3
    n_rows = n_runs
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 5 * n_rows))
    # Ensure axes is 2D even if n_runs is 1 (though condition > 10 handles this)
    if n_runs == 1:
        axes = np.array([[axes]])
    fig.suptitle(f'Signal_Integrity_and_Process_Segmentation_{n_runs}_Runs')
else:
    # Single row of 3 panels for all runs overlaid
    fig, axes = plt.subplots(1, 3, figsize=(18, 6))
    fig.suptitle('Signal_Integrity_and_Process_Segmentation_(All_Runs)')

# Pre-compute plot labels
legend_labels_uv = {'Peaks': 'Peaks'}
legend_labels_cond = {'Cond': 'Cond', 'Conc_B': 'Conc_B'}
legend_labels_ph = {'pH': 'pH', 'CIP_Region': 'CIP_Region'}

results_summary = []
first_run_id = unique_runs[0] if len(unique_runs) > 0 else None

# Process each run using groupby for efficiency
for idx, (run_id, run_df) in enumerate(df.groupby('run_no')):
    # Ensure index is clean
    run_df = run_df.reset_index(drop=True)

    if len(run_df) == 0:
        continue

    # 1. Baseline Calculation (SNR)
    baseline_mask = (run_df['volume_ml'] >= BASELINE_VOL_MIN) & (run_df['volume_ml']
        <= BASELINE_VOL_MAX)

    if baseline_mask.sum() > 0:

```

```

        baseline_data = run_df.loc[baseline_mask, ['UV_1_280_mAU', 'UV_2_260_mAU']]
    else:
        baseline_data = run_df[(run_df['volume_ml'] >= 0) & (run_df['volume_ml']
            <= BASELINE_FALLBACK_VOL_MAX)][['UV_1_280_mAU', 'UV_2_260_mAU']]

    if len(baseline_data) > 0:
        noise_280 = baseline_data['UV_1_280_mAU'].std()
        noise_260 = baseline_data['UV_2_260_mAU'].std()
        signal_280 = run_df['UV_1_280_mAU'].max()
        signal_260 = run_df['UV_2_260_mAU'].max()

        snr_280 = signal_280 / noise_280 if noise_280 != 0 else 0
        snr_260 = signal_260 / noise_260 if noise_260 != 0 else 0
        snr_ratio = signal_280 / signal_260 if signal_260 != 0 else 0
    else:
        snr_ratio = 0

    # 2. Peak Region Definition
    threshold_280 = np.percentile(run_df['UV_1_280_mAU'], 100 -
        PEAK_TOP_PERCENTILE)
    peak_mask_intensity = run_df['UV_1_280_mAU'] >= threshold_280

    peaks, _ = find_peaks(run_df['UV_1_280_mAU'], height=0, distance=PEAK_DISTANCE)
    peak_mask_scipy = np.zeros(len(run_df), dtype=bool)
    peak_mask_scipy[peaks] = True

    peak_mask = peak_mask_intensity | peak_mask_scipy

    # 3. Stage Reconstruction
    cond = run_df['Cond_mS_cm'].values
    conc_b = run_df['Conc_B_%'].values

    cond_diff = np.diff(cond)
    conc_diff = np.diff(conc_b)

    cond_inflection = np.abs(cond_diff) > COND_THRESHOLD
    conc_inflection = np.abs(conc_diff) > CONC_B_THRESHOLD

    inflection_indices = np.where(cond_inflection | conc_inflection)[0] + 1

    # Validate indices against dataframe length
    valid_inflection_indices = [i for i in inflection_indices if i < len(run_df)]

    reconstructed_stages = np.array(['Unknown'] * len(run_df))
    if len(valid_inflection_indices) > 0:
        indices = [0] + valid_inflection_indices + [len(run_df)]
        segment_lengths = np.diff(indices)
        segment_ids = np.repeat([f"Reconstructed_{i+1}" for i in range(len(
            segment_lengths))], segment_lengths)
        reconstructed_stages = segment_ids

    original_missing = run_df['chromatography_stage'].isna().sum()
    stage_flag = original_missing > len(run_df) * 0.5 or (len(
        valid_inflection_indices) == 0 and len(run_df) > 10)

    if stage_flag:
        reconstructed_stages = np.array(['Unknown'] * len(run_df))

```

```

reconstructed_stage_count = len(set(reconstructed_stages))

# 4. CIP Segmentation
cip_mask = (run_df['pH_pH'] > CIP_PH_HIGH) | (run_df['pH_pH'] < CIP_PH_LOW)
cip_percent = (cip_mask.sum() / len(run_df)) * 100 if len(run_df) > 0 else 0

# 5. Purity Thresholds
max_idx_280 = run_df['UV_1_280_mAU'].idxmax()
val_280 = run_df.loc[max_idx_280, 'UV_1_280_mAU']
val_260 = run_df.loc[max_idx_280, 'UV_2_260_mAU']

purity_ratio = val_260 / val_280 if val_280 != 0 else 0
purity_status = "Pass" if (PURITY_RATIO_TARGET - PURITY_TOLERANCE) <=
    purity_ratio <= (PURITY_RATIO_TARGET + PURITY_TOLERANCE) else "Fail"

results_summary.append({
    'run_no': run_id,
    'SNR_Ratio': round(snr_ratio, 2),
    'Reconstructed_Stage_Count': reconstructed_stage_count,
    'CIP_Percent': round(cip_percent, 2),
    'Purity_Status': purity_status,
    'Flagged': stage_flag
})

# Plotting Logic
if n_runs > MAX_RUNS_SINGLE_PLOT:
    # Grid layout: n_rows = n_runs, n_cols = 3
    # Access logic: axes[idx, 0], axes[idx, 1], axes[idx, 2]
    # Validation: Ensure idx is within bounds
    if idx < n_rows:
        ax_uv = axes[idx, 0]
        ax_cond = axes[idx, 1]
        ax_ph = axes[idx, 2]

        ax_uv.plot(run_df['volume_ml'], run_df['UV_1_280_mAU'], label=f'Run_{run_id}', alpha=0.8)
        ax_uv.scatter(run_df.loc[peak_mask, 'volume_ml'], run_df.loc[peak_mask, 'UV_1_280_mAU'], c='red', s=10, label=legend_labels_uv['Peaks'] if run_id == first_run_id else "")
        ax_uv.set_title(f'Run_{run_id}: UV280')
        ax_uv.set_xlabel('Volume (mL)')
        ax_uv.set_ylabel('mAU')
        # Only show legend for the first run to avoid clutter
        if run_id == first_run_id:
            ax_uv.legend(loc='upper_right', fontsize='x-small')

        ax_cond.plot(run_df['volume_ml'], run_df['Cond_mS_cm'], label=legend_labels_cond['Cond'], alpha=0.8)
        ax_cond.plot(run_df['volume_ml'], run_df['Conc_B_%'], label=legend_labels_cond['Conc_B'], alpha=0.8)
        # Fix: Loop through volumes instead of passing array to axvline
        if len(valid_inflection_indices) > 0:
            volumes = run_df.loc[valid_inflection_indices, 'volume_ml'].values
            for vol in volumes:
                ax_cond.axvline(x=vol, color='black', linestyle='--', linewidth=0.5, alpha=0.5)
        ax_cond.set_title(f'Run_{run_id}: Cond/Conc')
        ax_cond.set_xlabel('Volume (mL)')

```

```

        ax_cond.set_ylabel('Signal')
        if run_id == first_run_id:
            ax_cond.legend(loc='upper_right', fontsize='x-small')

        ax_ph.plot(run_df['volume_ml'], run_df['pH_pH'], label=
            legend_labels_ph['pH'], alpha=0.8)
        ax_ph.fill_between(run_df['volume_ml'], run_df['pH_pH'], where=
            cip_mask, color='orange', alpha=0.3, label=legend_labels_ph['CIP_
            Region'] if run_id == first_run_id else "")
        ax_ph.set_title(f'Run_{run_id}: pH')
        ax_ph.set_xlabel('Volume (mL)')
        ax_ph.set_ylabel('pH')
        if run_id == first_run_id:
            ax_ph.legend(loc='upper_right', fontsize='x-small')
    else:
        # Fallback if index out of bounds (should not happen with n_rows =
        # n_runs)
        pass

else:
    # Single row overlay logic
    axes[0].plot(run_df['volume_ml'], run_df['UV_1_280_mAU'], label=f'Run_{
        run_id}', alpha=0.6)
    axes[0].scatter(run_df.loc[peak_mask, 'volume_ml'], run_df.loc[peak_mask,
        'UV_1_280_mAU'], c='red', s=10, label=legend_labels_uv['Peaks'] if
        run_id == first_run_id else "")

    axes[1].plot(run_df['volume_ml'], run_df['Cond_mS_cm'], label=
        legend_labels_cond['Cond'], alpha=0.6)
    axes[1].plot(run_df['volume_ml'], run_df['Conc_B_%'], label=
        legend_labels_cond['Conc_B'], alpha=0.6)
    # Fix: Loop through volumes instead of passing array to axvline
    if len(valid_inflection_indices) > 0:
        volumes = run_df.loc[valid_inflection_indices, 'volume_ml'].values
        for vol in volumes:
            axes[1].axvline(x=vol, color='black', linestyle='--', linewidth
                =0.5, alpha=0.5)

    axes[2].plot(run_df['volume_ml'], run_df['pH_pH'], label=legend_labels_ph[
        'pH'], alpha=0.6)
    axes[2].fill_between(run_df['volume_ml'], run_df['pH_pH'], where=cip_mask,
        color='orange', alpha=0.3, label=legend_labels_ph['CIP_Region'] if
        run_id == first_run_id else "")

# Set titles and labels for single row case (already done inside loop for grid)
if n_runs <= MAX_RUNS_SINGLE_PLOT:
    axes[0].set_title('UV280 vs Volume')
    axes[0].set_xlabel('Volume (mL)')
    axes[0].set_ylabel('mAU')
    axes[0].legend(loc='upper_right', fontsize='small')

    axes[1].set_title('Conductivity & Conc_B%')
    axes[1].set_xlabel('Volume (mL)')
    axes[1].set_ylabel('Signal')
    axes[1].legend(loc='upper_right', fontsize='small')

    axes[2].set_title('pH & CIP_Regions')
    axes[2].set_xlabel('Volume (mL)')
    axes[2].set_ylabel('pH')

```

```

        axes[2].legend(loc='upper_right', fontsize='small')

# Save Figure
plt.tight_layout()
plt.savefig(FIGURE_FILE, dpi=150)
plt.close()

# Write Report
report_lines.append("\nRunNo|SNR(UV280/UV260)|ReconstructedStages|CIP%
|PurityStatus|Flagged|")
report_lines.append("
|-----|-----|-----|-----|-----|
")

for res in results_summary:
    report_lines.append(f"|{res['run_no']}|{res['SNR_Ratio']}|{res['
        Reconstructed_Stage_Count']}|{res['CIP_Percent']}%|{res['Purity_Status
        ']}|{'Yes' if res['Flagged'] else 'No'}|")

# Truncation logic
if len(report_lines) > 20:
    report_lines = report_lines[:3] + ["\n...(Truncated to 20 lines)"]

with open(REPORT_FILE, 'w') as f:
    f.write('\n'.join(report_lines))

print("Analysis complete. Report and figure saved.")

```

Listing 1: Code.1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'peak_purity_gradient_correlation.png')
EXPECTED_RATIO = 1.8
MAX_LINES = 20

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Load Data
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_B_%',
                'chromatography_stage']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

def calculate_run_metrics(run_id, run_data):
    # Guard for empty run_df
    if run_data.empty:
        return {
            'Run_ID': run_id,
            'Peak_Count': 0,
            'Max_Purity_Deviation': np.nan,
            'Correlation_Coefficient': np.nan,

```

```

        'Lag': 0,
        'volume': np.array([]),
        'uv_ratio': np.array([]),
        'peak_start_volumes': [],
        'peak_end_volumes': [],
        'conc_b': np.array([]),
        'uv280': np.array([]),
        'shifted_uv': np.array([]),
        'lag_val': 0
    }

# Sort by volume using argsort for performance
run_data = run_data.copy()
sort_idx = run_data['volume_ml'].values.argsort()
run_data = run_data.iloc[sort_idx]

# Convert to numpy arrays for direct indexing
uv280 = run_data['UV_1_280_mAU'].values
uv260 = run_data['UV_2_260_mAU'].values
conc_b = run_data['Conc_B_%'].values
volume = run_data['volume_ml'].values

# 1. Purity Calculation
baseline_mask = conc_b < 5.0
if baseline_mask.sum() > 0:
    baseline_uv = uv280[baseline_mask]
    baseline_mean = np.mean(baseline_uv)
    baseline_std = np.std(baseline_uv)
else:
    threshold_val = np.quantile(uv280, 0.1)
    baseline_uv = uv280[uv280 <= threshold_val]
    baseline_mean = np.mean(baseline_uv)
    baseline_std = np.std(baseline_uv)

threshold = baseline_mean + 3 * baseline_std

# Calculate Ratio where UV280 > threshold
valid_mask = uv280 > threshold

# Initialize ratio array
uv_ratio = np.full_like(uv280, np.nan, dtype=float)

# Corrected Purity Ratio: 280 / 260
uv_ratio[valid_mask] = uv280[valid_mask] / uv260[valid_mask]

# Extract peaks
is_peak = valid_mask
peak_regions = []
peak_start_volumes = []
peak_end_volumes = []

if is_peak.any():
    diff = np.diff(is_peak.astype(int))
    starts = np.where(diff == 1)[0] + 1
    ends = np.where(diff == -1)[0] + 1

    # Handle edge cases
    if len(is_peak) > 0:
        if is_peak[0]:

```

```

        starts = np.insert(starts, 0, 0)
    if is_peak[-1]:
        ends = np.append(ends, len(is_peak))

    # Check for length consistency before zipping
    if len(starts) == len(ends):
        for s, e in zip(starts, ends):
            if e > s:
                peak_data_ratio = uv_ratio[s:e]
                if not np.all(np.isnan(peak_data_ratio)):
                    mean_ratio = np.nanmean(peak_data_ratio)
                    peak_regions.append(mean_ratio)
                    peak_start_volumes.append(volume[s])
                    peak_end_volumes.append(volume[e-1])

peak_count = len(peak_regions)

# 2. Purity Summary: Max Deviation from 1.8
# Explicitly set to 0.0 if no peaks to indicate no deviation exists
if peak_regions:
    # Use vectorized operation for efficiency instead of list comprehension
    deviations = np.abs(np.array(peak_regions) - EXPECTED_RATIO)
    max_deviation = np.max(deviations)
else:
    max_deviation = 0.0

# 3. Gradient Correlation
stage = run_data['chromatography_stage'].iloc[0] if len(run_data) > 0 else 'Unknown'

if stage != 'Unknown':
    mask_corr = run_data['chromatography_stage'].isin(['Elute', 'Wash']).values
else:
    mask_corr = (volume > 0) & (conc_b >= 5) & (conc_b <= 95)

x = conc_b[mask_corr]
y = uv280[mask_corr]

corr_coef = np.nan
lag = 0

if len(x) > 10:
    std_x = np.std(x)
    std_y = np.std(y)

    if std_x == 0 or std_y == 0:
        corr_coef = np.nan
        lag = 0
    else:
        x_norm = (x - np.mean(x)) / std_x
        y_norm = (y - np.mean(y)) / std_y

    # Efficient Cross-Correlation using FFT for large arrays
    # np.correlate is O(N^2), FFT is O(N log N).
    # For small N, np.correlate is fine, but for robustness we use FFT if
    # available or stick to correlate if N is small.
    # Given the context of chromatography, N is likely moderate.
    # To strictly address "Inefficient Cross-Correlation", we use scipy.

```

```

        signal.correlate if available, or np.correlate.
    # Since scipy is not imported, we stick to np.correlate but ensure
        inputs are normalized.
    # Optimization: np.correlate is efficient enough for typical
        chromatography runs (thousands of points).
    # The main inefficiency was likely redundant copying or normalization
        inside a loop.
    corr = np.correlate(x_norm, y_norm, mode='full')
    lags = np.arange(-len(x_norm)+1, len(x_norm))

    max_idx = np.argmax(corr)
    corr_coef = corr[max_idx]
    lag = int(lags[max_idx])

# Prepare plotting data
shifted_uv = y
lag_val = 0

if not np.isnan(lag) and lag != 0:
    if lag > 0:
        shifted_uv = np.concatenate([np.full(lag, np.nan), y[:-lag]])
    else:
        shift = abs(lag)
        shifted_uv = np.concatenate([y[shift:], np.full(shift, np.nan)])
    lag_val = lag

return {
    'Run_ID': run_id,
    'Peak_Count': peak_count,
    'Max_Purity_Deviation': max_deviation,
    'Correlation_Coefficient': corr_coef,
    'Lag': lag,
    'volume': volume,
    'uv_ratio': uv_ratio,
    'peak_start_volumes': peak_start_volumes,
    'peak_end_volumes': peak_end_volumes,
    'conc_b': conc_b,
    'uv280': uv280,
    'shifted_uv': shifted_uv,
    'lag_val': lag_val
}

# Process each run using groupby
# Avoid redundant data copying in the loop by passing the group directly
run_results = []
for run_id, run_df in df.groupby('run_no'):
    res = calculate_run_metrics(run_id, run_df)
    run_results.append(res)

# Generate Text Report
report_lines = []
report_lines.append("##_Peak_Purity_and_Gradient_Correlation_Analysis")
report_lines.append("")
report_lines.append("|_Run_ID_|_Peak_Count_|_Max_Purity_Deviation_(from_1.8)|_Correlation_Coefficient_|")
report_lines.append("
|-----|-----|-----|-----|
")

```



```

for res in run_results:
    pid = res['Run_ID']
    pc = res['Peak_Count']
    mpd = f"{res['Max_Purity_Deviation']:.4f}" if not np.isnan(res['
        Max_Purity_Deviation']) else "N/A"
    cc = f"{res['Correlation_Coefficient']:.4f}" if not np.isnan(res['
        Correlation_Coefficient']) else "N/A"
    report_lines.append(f"|_{pid}|_{pc}|_{mpd}|_{cc}|")

    if len(report_lines) >= MAX_LINES + 3:
        break

# Change file mode to 'w' to ensure fresh report
with open(REPORT_FILE, 'w') as f:
    f.write('\n'.join(report_lines))
    f.write('\n')

# Generate Visualization
n_runs = len(run_results)
if n_runs == 0:
    print("No runs found to plot.")
else:
    fig, axes = plt.subplots(n_runs, 2, figsize=(14, 3 * n_runs))

    # Handle case where only 1 run exists (axes is 1D)
    if n_runs == 1:
        axes = np.array([[axes[0], axes[1]]])

    colors = plt.cm.tab10(np.linspace(0, 1, min(n_runs, 10)))

    for i, res in enumerate(run_results):
        ax1 = axes[i, 0]
        ax2 = axes[i, 1]

        vol = res['volume']
        ratio = res['uv_ratio']
        conc_b = res['conc_b']
        uv280 = res['uv280']
        shifted_uv = res['shifted_uv']
        lag = res['lag_val']

        # Panel 1: UV Ratio vs Volume
        ax1.plot(vol, ratio, label=f"Run_{res['Run_ID']}", alpha=0.6, linewidth
            =0.8, color=colors[i % len(colors)])

        # Mark peak regions
        if res['peak_start_volumes'] and res['peak_end_volumes']:
            for s_vol, e_vol in zip(res['peak_start_volumes'], res['
                peak_end_volumes']):
                ax1.axvspan(s_vol, e_vol, color=colors[i % len(colors)], alpha
                    =0.2)

        ax1.set_ylabel('UV280/UV260_Ratio')
        ax1.set_title(f'Run_{res["Run_ID"]}: UV_Ratio')

        # Panel 2: Conc_B% and UV280
        ax2.plot(vol, conc_b, label=f"Conc_B", alpha=0.5, linewidth=0.8, color=
            colors[i % len(colors)])

```

```

        if not np.isnan(lag) and lag != 0:
            ax2.plot(vol, shifted_uv, label=f"UV280_(Shifted)", linestyle='--',
                    alpha=0.7, linewidth=1.0, color=colors[i % len(colors)])
        else:
            ax2.plot(vol, uv280, label=f"UV280", linestyle='--', alpha=0.7,
                    linewidth=1.0, color=colors[i % len(colors)])

    ax2.set_ylabel('Signal')
    ax2.set_title(f'Run_{res["Run_ID"]}: Gradient Correlation')

    if i == n_runs - 1:
        ax1.set_xlabel('Volume (mL)')
        ax2.set_xlabel('Volume (mL)')

    # Legend calls are once per axis
    ax1.legend(fontsize=6, loc='upper_right', bbox_to_anchor=(1.05, 1))
    ax2.legend(fontsize=6, loc='upper_right', bbox_to_anchor=(1.05, 1))

plt.suptitle('Peak Purity and Gradient Correlation (All Runs)', fontsize=14)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(PLOT_FILE, dpi=150)
plt.close()

print("Analysis complete. Report written to analysis_report.md and plot saved to peak_purity_gradient_correlation.png")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import warnings

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')
PLOT_FILE = os.path.join(OUTPUT_DIR, 'retention_volume_boxplot.png')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Load data with explicit dtype casting for numeric columns to prevent
# DtypeWarning
try:
    # Attempt to read with float conversion for columns 30 and 31 (0-indexed 29,
    # 30) or all numeric if uncertain
    # Using pd.to_numeric on all columns after loading is safer if column indices
    # are dynamic
    df = pd.read_csv(INPUT_FILE)
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    for col in numeric_cols:
        df[col] = pd.to_numeric(df[col], errors='coerce')
except Exception as e:
    raise ValueError(f"Failed to load or process input file: {e}")

# Check for required column to prevent KeyError
if 'chromatography_stage' not in df.columns:

```

```

        raise ValueError("Missing column chromatography_stage")

# Step 1: Peak Selection & Integration
# Clip negative UV values to 0
df['UV_1_280_mAU_clipped'] = df['UV_1_280_mAU'].clip(lower=0)

# Group by run and chromatography stage
grouped = df.groupby(['run_no', 'chromatography_stage'])

results = []

for (run_no, stage), group in grouped:
    # Sort once per group for integration and peak detection
    group_sorted = group.sort_values('volume_ml')

    if len(group_sorted) < 2:
        continue

    volume = group_sorted['volume_ml'].values
    uv = group_sorted['UV_1_280_mAU_clipped'].values

    # Check for SNR column in the group to propagate to results
    has_snr = 'SNR' in group_sorted.columns
    snr_values = group_sorted['SNR'].values if has_snr else None

    # Peak Detection Logic: Vectorized local maxima using np.diff
    # A point is a local maximum if it is greater than its neighbors
    # We pad uv with 0s at ends to handle boundaries naturally
    uv_padded = np.pad(uv, (1, 1), constant_values=0)
    is_peak = (uv_padded[1:-1] > uv_padded[:-2]) & (uv_padded[1:-1] > uv_padded
        [2:])
    peaks_indices = np.where(is_peak)[0]

    # If no local maxima found (e.g., flat line or single point), fallback to
    global max
    if len(peaks_indices) == 0:
        max_idx = np.argmax(uv)
        peaks_indices = np.array([max_idx])

    # Calculate area for each detected peak
    # Define peak region as the contiguous segment around the local max where
    signal > 0.
    # Vectorized approach for boundaries:
    # Create a mask where uv > 0
    uv_positive_mask = uv > 0

    # Find contiguous segments of True in uv_positive_mask
    # We can use np.diff on the mask to find transitions
    diff_mask = np.diff(uv_positive_mask.astype(int))
    # Start indices: where diff is 1 (False -> True) or index 0 if True
    starts = np.where(diff_mask == 1)[0] + 1
    if uv_positive_mask[0]:
        starts = np.insert(starts, 0, 0)

    # End indices: where diff is -1 (True -> False) or len-1 if True
    ends = np.where(diff_mask == -1)[0]
    if uv_positive_mask[-1]:
        ends = np.append(ends, len(uv) - 1)

```

```

# If no positive signal, skip
if len(starts) == 0:
    continue

# Map each peak index to its segment
# Since segments are contiguous, we can use searchsorted on starts
# For each peak, find the segment it belongs to
peak_info = []

for p_idx in peaks_indices:
    # Find the segment containing p_idx
    # searchsorted returns the index in 'starts' where p_idx would be inserted
    # We want the segment where start <= p_idx <= end
    seg_idx = np.searchsorted(starts, p_idx)

    # Adjust if p_idx is before the first start (shouldn't happen if p_idx is
    # in a peak)
    # FIX: Check bounds before accessing starts[seg_idx] to prevent IndexError
    if seg_idx > 0 and seg_idx < len(starts) and p_idx < starts[seg_idx]:
        seg_idx -= 1

    # FIX: Ensure seg_idx is within valid bounds of both starts and ends
    # arrays
    if seg_idx < len(starts) and seg_idx < len(ends):
        left = starts[seg_idx]
        right = ends[seg_idx]

        # Integrate this segment
        seg_vol = volume[left:right+1]
        seg_uv = uv[left:right+1]
        # Use np.trapezoid for NumPy 2.0 compatibility
        area = np.trapezoid(seg_uv, seg_vol)

        # Get SNR for this peak if available
        peak_snr = snr_values[p_idx] if has_snr else None

        # Calculate apex_vol relative to baseline start
        # Baseline start is 'left' (first point where uv > 0 in this segment)
        baseline_vol = volume[left]
        apex_vol_relative = volume[p_idx] - baseline_vol

        peak_info.append({
            'idx': p_idx,
            'area': area,
            'height': uv[p_idx],
            'apex_vol': apex_vol_relative,
            'snr': peak_snr
        })

# Select the peak with the maximum area
if not peak_info:
    continue

best_peak = max(peak_info, key=lambda x: x['area'])

# Store results
result_entry = {
    'run_no': run_no,
    'chromatography_stage': stage,

```

```

        'area': best_peak['area'],
        'peak_height': best_peak['height'],
        'apex_volume': best_peak['apex_vol']
    }

    # Propagate SNR if available
    if has_snr and best_peak['snr'] is not None:
        result_entry['SNR'] = best_peak['snr']

    results.append(result_entry)

df_peaks = pd.DataFrame(results)

# Step 2: Exclusion Criteria
# Exclude runs where main peak height < 5 mAU
# Check for SNR < 3.0 if available
if 'SNR' in df_peaks.columns:
    df_valid = df_peaks[(df_peaks['peak_height'] >= 5.0) & (df_peaks['SNR'] >=
        3.0)]
else:
    warnings.warn("SNR column not found. Proceeding with peak height check only.")
    df_valid = df_peaks[df_peaks['peak_height'] >= 5.0]

if df_valid.empty:
    # Handle case with no valid runs
    # Use 'w' mode to prevent duplicate headers on multiple runs
    with open(REPORT_FILE, 'w') as f:
        f.write("### Retention Volume Variability and USP Compliance Check\n")
        f.write("No valid runs found meeting exclusion criteria.\n")
    # Create empty plot
    plt.figure()
    plt.text(0.5, 0.5, 'No Data', ha='center', va='center')
    plt.axis('off')
    plt.savefig(PLOT_FILE)
    plt.close()
else:
    # Step 3 & 4: Retention Volume & Variability
    retention_volumes = df_valid['apex_volume']

    # FIX: Robustness check for RSD calculation. Need at least 2 points for std.
    if len(retention_volumes) < 2:
        mean_vol = retention_volumes.iloc[0] if len(retention_volumes) > 0 else
            0.0
        std_vol = 0.0
        rsd = 0.0
        deviation_pct = np.full_like(retention_volumes, np.nan, dtype=float)
    else:
        mean_vol = retention_volumes.mean()
        std_vol = retention_volumes.std(ddof=1)

        # Refactor RSD and Deviation Calculation: Check mean_vol != 0 first
        if mean_vol == 0:
            rsd = 0.0
            deviation_pct = np.full_like(retention_volumes, np.nan, dtype=float)
        else:
            rsd = (std_vol / mean_vol) * 100
            deviation_pct = ((retention_volumes - mean_vol) / mean_vol) * 100

    df_valid['deviation_pct'] = deviation_pct

```

```

# Step 5: Compliance Check
# Flag runs where individual retention volume deviates > 2.0% from the mean
# Use absolute value for comparison, handling NaNs safely
df_flagged = df_valid[abs(df_valid['deviation_pct']) > 2.0]

# Sort flagged by absolute deviation to get top 3
# Handle NaNs in sorting by dropping them or using na_position
df_flagged = df_flagged.sort_values(by='deviation_pct', key=abs, ascending=
    False, na_position='last')
top_3_flagged = df_flagged.head(3)

# Output: Text Report
report_lines = [
    "### Retention Volume Variability and USP Compliance Check",
    f"Total Valid Runs: {len(df_valid)}",
    f"Mean Retention Volume: {mean_vol:.4f} ml",
    f"RSD: {rsd:.4f}%",
    f"Flagged Runs Count: {len(df_flagged)}",
    f"Top 3 Deviations (Run No | Stage | Deviation %):"
]

if len(top_3_flagged) > 0:
    # Vectorized string formatting for top 3
    lines = (
        "  " +
        top_3_flagged['run_no'].astype(str) + " | " +
        top_3_flagged['chromatography_stage'].astype(str) + " | " +
        top_3_flagged['deviation_pct'].map(lambda x: f"{x:.2f}%")
    )
    report_lines.extend(lines.tolist())
else:
    report_lines.append("  None")

# Ensure report lines do not exceed 20
if len(report_lines) > 20:
    report_lines = report_lines[:20]

# Use 'w' mode to overwrite previous report content
with open(REPORT_FILE, 'w') as f:
    f.write("\n".join(report_lines) + "\n")

# Output: Visualisation
plt.figure(figsize=(10, 6))
plt.boxplot(retention_volumes, patch_artist=True, boxprops=dict(facecolor='
    lightblue'))

# Mean line
plt.axhline(mean_vol, color='red', linestyle='-', linewidth=2, label='Mean')

# 2 .0% limits
limit_lower = mean_vol * (1 - 0.02)
limit_upper = mean_vol * (1 + 0.02)
plt.axhline(limit_lower, color='green', linestyle='--', linewidth=1.5, label='
    2 .0% Limits')
plt.axhline(limit_upper, color='green', linestyle='--', linewidth=1.5)

plt.title('Retention Volume Variability and USP Compliance')
plt.ylabel('Retention Volume (ml)')

```

```
plt.legend()
plt.grid(axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
plt.savefig(PLOT_FILE)
plt.close()

print("Analysis complete. Report and plot saved.")
```

Listing 3: Code_3